

Life Cycle , Functional Vs Class Component And Hooks In React Js

By Sir Yasir Ali Lashari

Lifecycle Methods in React

In React, every component follows a lifecycle. It describes how a component is created, updated, and removed.

Phases of Lifecycle

1. Mounting

This is the stage when a component appears on the screen for the first time.
Example: Opening an application or loading a page.

2. Updating

This stage occurs when the data of a component changes and the screen updates. Example: Clicking a button and the number changes.

3. Unmounting

This is the stage when a component is removed from the screen.
Example: Closing a page or navigating to another screen.

Class Components

Class components are components created using ES6 class syntax. They were used in earlier versions of React to manage state and behavior.

Example:

```
import React, { Component } from "react";

class Hello extends Component {
  render() {
    return <h1>Hello World</h1>;
  }
}

export default Hello;
```

Key Points:

- Uses class
- Must have render() method
- Returns JSX inside render()

Functional Components

Functional components are simple JavaScript functions that return JSX. They are the modern way of creating components in React.

Example:

```
function Hello() {  
  return <h1>Hello World</h1>;  
}
```

```
export default Hello;
```

Key Points:

- Just a simple function
- Directly returns JSX
- No render() needed

Class vs Functional Components

Class Components

- Use class syntax
- More complex structure
- Use this keyword
- Manage data using state

Functional Components

- Simple and clean syntax
- Easy to understand
- No use of this
- Use Hooks for state and logic

Real Life Example

Class Component: Like a manual car. It requires more effort to control.

Functional Component: Like an automatic car. It is easy to use.

Why Functional Components are Preferred

- Easier to write and understand
- Less code required
- No confusion of this keyword

- Hooks provide powerful features
- Better readability and maintenance

Hooks in React

Hooks are functions that allow functional components to use state and lifecycle features.

Common Hooks:

- useState
- useEffect
- useRef

Why Hooks are Used When Props Exist

Props and Hooks have different purposes.

Props:

- Used to pass data from one component to another
- Read-only

Hooks:

- Used to manage data inside a component
- Data can be updated

Props bring data into a component, while Hooks control and update that

data. **Class Component with State**

```
import React, { Component } from "react";

class Counter extends Component {
  constructor() {
    super();

    // this refers to the current object that is using the code
    // (current instance)
    // In React: this.state means state of this component

    this.state = { count: 0 };
  }

  increase = () => {
```

```

    this.setState({ count: this.state.count + 1 });
  };

  render() {
    return (
      <>
        <h1>{this.state.count}</h1>
        <button onClick={this.increase}>Increase</button> </>
      </>
    );
  }
}

export default Counter;

```

Functional Component with State

```

import { useState } from "react";

function Counter() {
  const [count, setCount] = useState(0);

  return (
    <>
      <h1>{count}</h1>
      <button onClick={() => setCount(count + 1)}>
Increase
      </button>
    </>
  );
}

export default Counter;

```

Understanding useState in React

In React, `useState` is a Hook used to store and manage data inside functional components.

Syntax:

```
const [state, setState] = useState(initialValue);
```

1. `useState(0)`

- `useState(0)` sets the initial value of the state

- 0 is the starting value
- It tells React: start the value from 0

2. count

- count is the current state value
- It stores the data that can change over time
- Example values: 0, 1, 2, 3...

3. setCount

- setCount is a function used to update the state
- When called, it changes count and re-renders the component

Example:

```
setCount(count + 1);
```

4. Can we change names?

Yes, names can be changed.

Example:

```
const [number, setNumber] = useState(0);
```

Rules: - First value = state variable (current value) - Second value = function to update state

[Seek help through patience and prayer. (Surah Al-Baqarah 2:45)]